Java Multi-threading

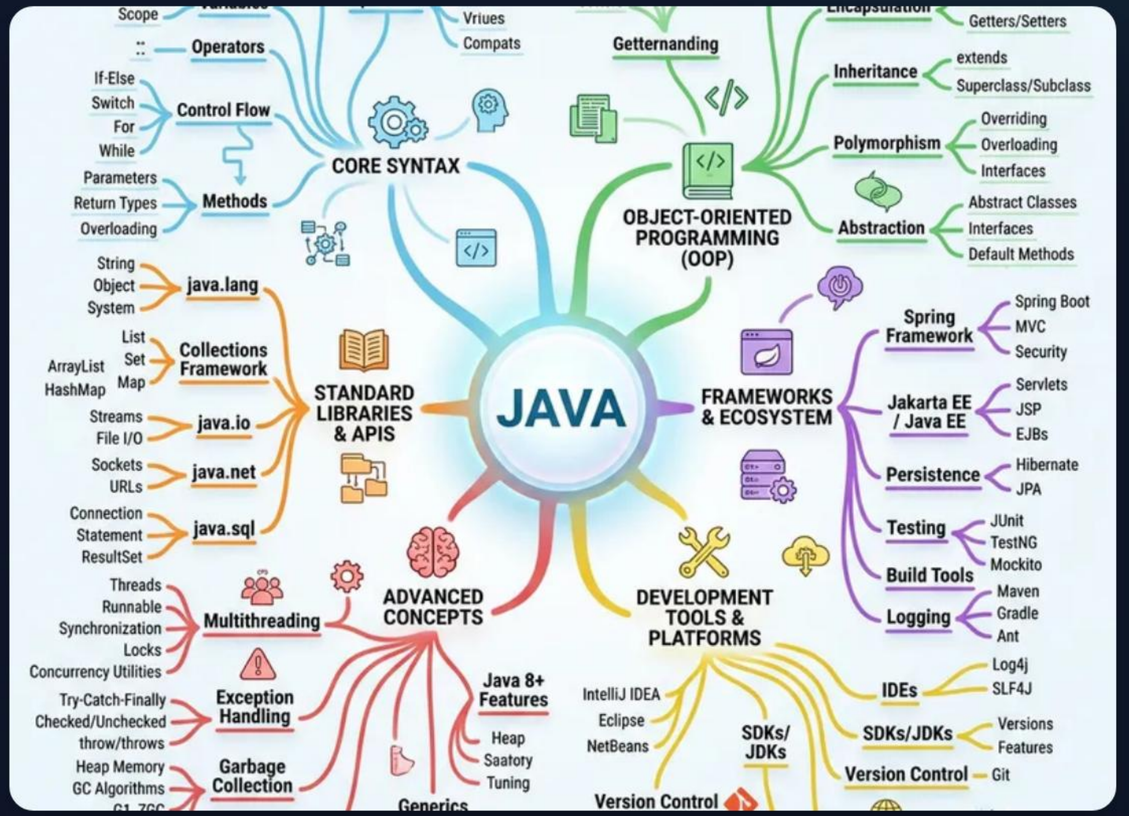
Phase 2: Thread Creation & Phase 3: Core Methods

Comprehensive Guide to Concurrent Programming

WHAT IS A THREAD?

A **Thread** is the smallest unit of execution within a process. Java provides built-in support for multithreaded programming.

- Path of execution in a program.
- Lightweight compared to OS processes.
- Threads share the same memory space (Heap).
- Main thread is started by the JVM automatically.
- Essential for asynchronous tasks & UI responsiveness.



THE 6 LIFECYCLE STATES

NEW

Thread object created but `start()` not yet called.

RUNNABLE

Thread is ready to run or currently executing in JVM.

BLOCKED

Waiting to acquire a monitor lock to enter/re-enter synchronized block.

WAITING

Waiting indefinitely for another thread to perform an action.

TIMED WAITING

Waiting for a specified time (e.g., `sleep`, `join` with timeout).

TERMINATED

Thread has completed execution or crashed.

METHOD 1: EXTENDING THREAD

</>

Implementation Steps

1. Extend `java.lang.Thread` class.
2. Override the `run()` method.
3. Instantiate your class.
4. Invoke `start()`.

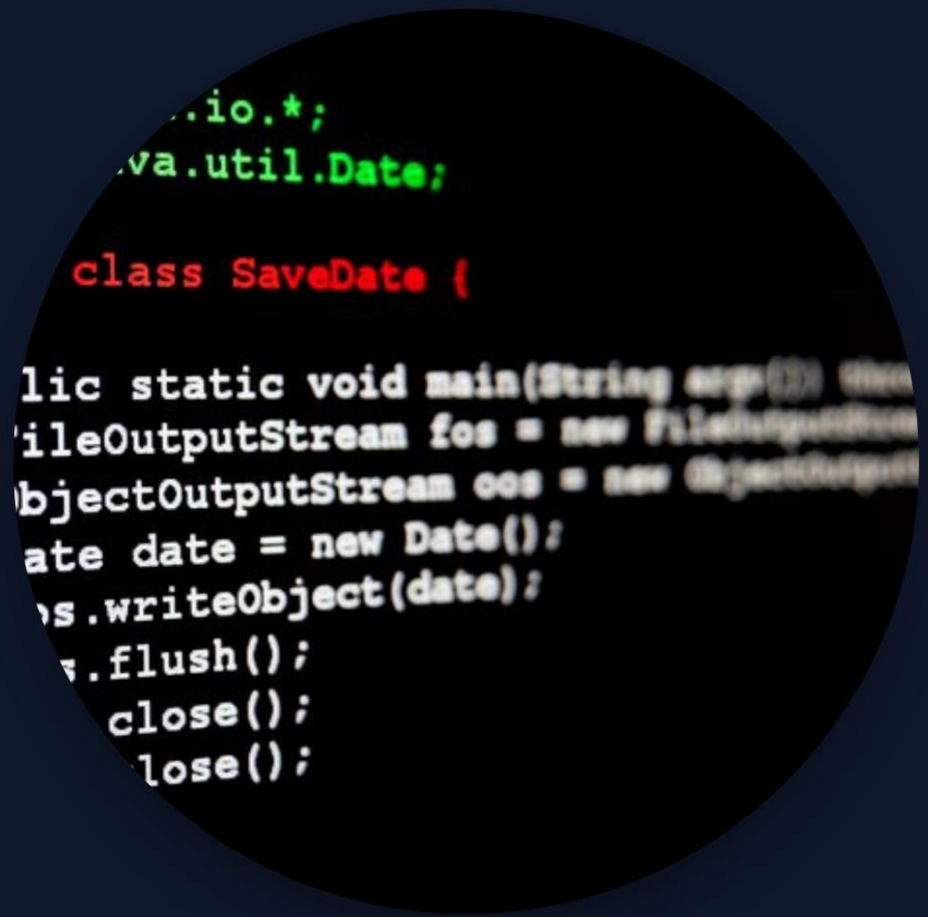
⚠

The Constraint

Since Java does not support multiple inheritance, your class cannot extend any other class once it extends `Thread`.

Use only if you plan to override other Thread methods.

METHOD 2: IMPLEMENTING RUNNABLE



Why Runnable is Better

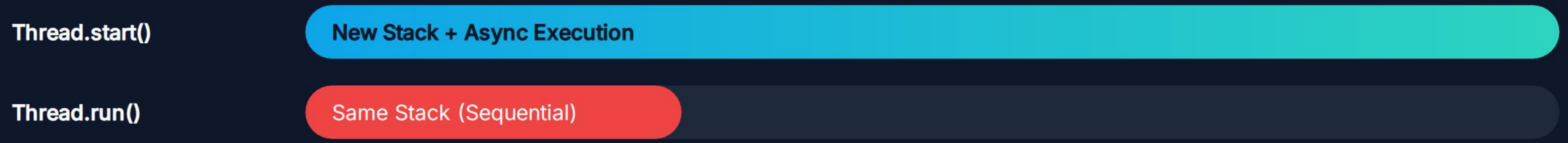
- **Flexibility:** Allows your class to extend another class while still being a Thread.
- **Decoupling:** Separates the Task (Runnable) from the Runner (Thread).
- **Resource Sharing:** Better suited for sharing a single object instance among multiple threads.
- **Lambda Support:** Can be implemented as a simple Lambda expression since Java 8.

THREAD VS. RUNNABLE

Feature	Extending Thread Class	Implementing Runnable
Inheritance	Limited (cannot extend others)	Flexible (can extend others)
Memory	Each thread creates unique object	Threads can share same object
Complexity	Simple for small tasks	Recommended for robust apps
Override	Overwrites Thread behavior	Only implements the Task

THE BIG DISTINCTION: START() VS. RUN()

Calling `run()` directly is a common mistake. It does NOT create a new thread; it executes on the current stack.



- *`start()`: Registers the thread with the Thread Scheduler.*
- *`run()`: Just a normal method call. No multithreading occurs!*

THREAD MANAGEMENT: SLEEP & YIELD

sleep(long millis)

Causes the current thread to suspend execution for a specified period.

- Static method of Thread.
- Does NOT release locks.
- Throws `InterruptedException`.

yield()

A hint to the scheduler that the current thread is willing to yield its current use of a processor.

- Static method.
- Scheduler is free to ignore this hint.
- Rarely used in production code.

JOIN() & CURRENTTHREAD()

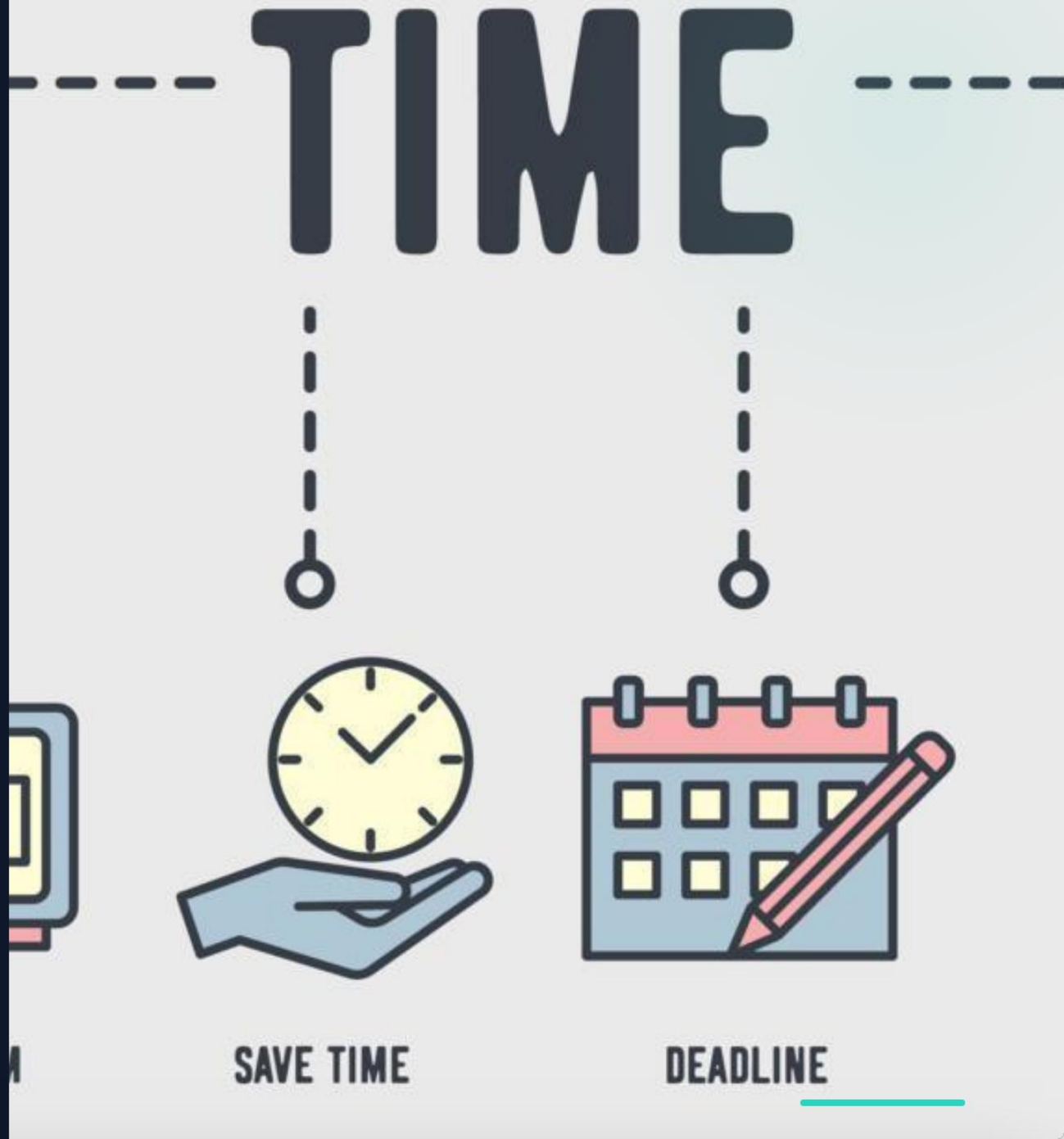
join() Method

Allows one thread to wait for the completion of another. If T1 calls `T2.join()`, T1 stops until T2 finishes.

currentThread()

Static method that returns a reference to the currently executing thread object.

```
Thread t = Thread.currentThread();
```



METADATA: NAMING & PRIORITY

1-10

PRIORITY RANGE

- **Naming:** Use `setName()` and `getName()`. Vital for debugging.
- **MIN_PRIORITY:** Value is 1.
- **NORM_PRIORITY:** Value is 5 (Default).
- **MAX_PRIORITY:** Value is 10.
- *Note: Higher priority doesn't guarantee earlier execution (Platform dependent).*

HOW TO PROPERLY STOP A THREAD

Never use `stop()`: It is deprecated because it is inherently unsafe (unlocks all monitors, causing inconsistent state).

Volatile Flag

Use a boolean `isRunning` flag. The thread checks the flag in its loop.

Interruption

Call `thread.interrupt()`. This sets the interrupt flag or wakes the thread from sleep.

Clean-up

Always release resources in a `finally` block when a thread terminates.

THE INTERRUPTION MECHANISM

InterruptedException

Thrown when a thread is interrupted while waiting/sleeping. It immediately clears the interrupt flag.

The Interrupt Flag

An internal status bit. Use it to gracefully ask a thread to stop working.

Method	Type	Functionality
`interrupt()`	Instance	Sets the interrupt status flag to **true**.
`isInterrupted()`	Instance	Checks the status of the flag. Does **not** clear it.
`interrupted()`	Static	Checks the status of the flag and **clears** it to false.

Questions & Answers

Thank you for your attention on Java Multi-threading Phases 2 & 3.

Happy Coding!

IMAGE SOURCES

https://media.easy-peasy.ai/0f6a3f77-b3bf-41dc-85e9-43648e0e0236/68bb6476-3a1f-4a9b-b17f-895785acb872_medium.webp

Source: easy-peasy.ai

https://media.hswstatic.com/eyJidWNrZXQiOiJjb250ZW50Lmhzd3N0YXRpYy5jb20iLCJrZXkiOiJnaWZcL2phdmEtY29kZS5qcGciLCJlZGl0cyl6eyJyZXNpemUiOnsid2lkdGgiOjgyOH19fQ==

Source: computer.howstuffworks.com

https://static.vecteezy.com/system/resources/previews/008/689/575/non_2x/time-banner-web-icon-fast-shipping-stopwatch-digital-alarm-save-time-deadline-hourglass-house-clock-time-management-illustration-concept-vector.jpg

Source: www.vecteezy.com